IoT Cat Feeder

An automatic, portion-controlled cat feeder with a web dashboard on the ESP8266

Project Proposal

Chindanai Jaiman 662115012
Bachelor of Science Software Engineering Program
College of Arts, Media, and Technology, Chiang Mai University

November 2026

Project Advisor
Asst.Prof.Phudinan Singkhamfu, Ph.D.

Table of Contents

Chapter 1 Introduction....3
1.1 Motivation....3
1.2 Purpose....3
1.3 Target Users and Intended Experience....3
1.4 Project Scope....4
1.5 Objectives....4
Chapter 2 Review and Analysis....5
2.1 Business and Software Review....5
2.2 Tool and Technology Review....5
2.3 Risk Analysis....6
Chapter 3 Product Perspective....7
3.1 Product Perspective....7
3.2 Product Features....7
3.3 System Architecture....7
3.4 Use Case Overview....7
Chapter 4 Quality Planning....9
Chapter 5 Limits, Milestones, and Deliverables....10
5.1 Limits....10
5.2 Milestones....10
5.3 Deliverable Items....10
5.4 Budget....10
5.5 Future Work....11

Chapter 1 Introduction

1.1 Motivation

A cat that is fed by hand eats at the convenience of the owner, not at a fixed time. When the owner leaves for work in the morning and returns in the evening, the usual workaround is to pour a large portion into the bowl and let the cat graze all day. Two problems follow. The food sits in the open bowl for ten hours or more, and the cat eats the entire portion before noon and then begs for the rest of the day.

The second problem is portion control. Dry food is calorie dense, and a cat that free feeds commonly gains weight. Veterinary guidance for an adult cat of average weight is a measured daily ration split across several meals, not an open bowl. An owner who is not at home at lunchtime cannot deliver the middle meal.

Commercial automatic feeders solve the timing problem, but most of them depend on a cloud service and a mobile app to program the schedule. If the vendor changes the app, or the household Wi-Fi is replaced, the device becomes a plastic box. For a student project the interesting engineering question is whether the same behaviour can be produced on a microcontroller that the owner can read, repair, and reflash.

1.2 Purpose

This project builds a cat feeder that dispenses dry food at three fixed times per day and also on demand from a phone. The device serves its own web dashboard over the local Wi-Fi network, so no account, no cloud service, and no vendor app are required. The owner opens a browser, sees the current state of the flap and the number of meals served today, and can trigger a feeding with one tap.

The project is also a study of the boundary between software and hardware. The interesting part is not the web page. It is the electrical and timing behaviour that the web page depends on: a servo that stalls and browns out the board, a clock that is wrong until the network answers, and a flap that must never stay open.

1.3 Target Users and Intended Experience

The system has two users with very different needs. The design is judged on whether both are served without one damaging the other.

1.3.1 Persona: The owner

Table 1. Owner persona

Attribute	Detail
Name	Nok
Age	29
Occupation	Office worker, Chiang Mai
Background	Lives in a rented room with one adult cat. Leaves at 07:30 and returns at 18:30. Uses a phone for everything. Has no tools beyond a screwdriver.
Goals	Feed the cat on a predictable schedule. Check from work that the morning meal was served. Add an extra meal when the cat begs.
Pain points	Cannot come home at lunch. Forgets whether the morning meal was served. Does not want another app account, and does not want to learn a configuration language.
Experience requirement	Open one page, read five values, tap one button. No login, no setup wizard, no pairing step.

1.3.2 Persona: The cat

Table 2. Cat persona and constraints

Attribute	Detail
Subject	One adult domestic cat, 4 kg
Behaviour	Eats at the sound of the flap. Approaches the device when it is near the bowl.
Constraint	The cat must not be able to open the flap with a paw, and must not be trapped by it.
Experience requirement	A meal arrives without the cat having to touch anything. The flap cycle is short enough not to startle the cat away from the bowl.

1.4 Project Scope

The scope is set so that the work is complete and demonstrable on 4 November 2026. Features that cannot be finished and tested by 2 November are moved to Future Work rather than shipped half-built.

Table 3. Scope of the delivered system

In scope	Out of scope
Scheduled feeding at three times per day using NTP time	Cloud storage and a hosted dashboard
On-demand feeding from a browser on the same network	A native mobile application
Flap status, daily meal count, last meal time and source	User accounts and authentication
Automatic flap close after a fixed open interval	Wet food dispensing
Cat presence detection with an ultrasonic sensor (target)	Camera or two-way audio
Portion control by weight with a load cell (target)	Support for more than one feeder or more than one cat

The two target features are carried as requirements so the design is ready for them, but they ship only if the parts arrive and pass test by 2 November. The reason for the rule is that a requirement that is written and tested earns credit, while a half-connected sensor does not.

1.5 Objectives

1. Deliver food at 08:00, 13:00, and 18:00 local time, using time obtained from a network time server rather than a countdown timer.
2. Serve a dashboard from the ESP8266 itself, reachable from a phone on the same Wi-Fi network in under two seconds.
3. Close the flap automatically after every feeding, in all code paths, including the path where the browser disconnects immediately after sending the command.
4. Record the time, count, and trigger source of each meal so the owner can confirm a feeding happened.
5. Detect a cat within 60 cm and dispense a meal in under two seconds, at most once per 30 minutes.
6. Dispense a portion within 15 percent of the target weight.
7. Document the design so another student can rebuild the device from the repository alone.

Chapter 2 Review and Analysis

2.1 Business and Software Review

The review below is written for a device that already exists in commercial form. The project is not trying to win on features. It is trying to win on independence, repairability, and cost.

2.1.1 Strengths

- **No cloud dependency.** The dashboard is served by the device. The system keeps working when the internet is down, and it works in a room with no router if the access point is provided by a phone.
- **Low cost.** The bill of materials is roughly one tenth of a mid-range commercial feeder, because there is no enclosure injection moulding, no app, and no subscription.
- **Measured portion instead of a timer.** Most low-cost feeders open a gate for a fixed time and trust gravity. A load cell turns the same mechanism into a closed loop, which is the difference between a timer and a controller.
- **Documented and repairable.** One firmware file, one wiring page, and one design document. A failed servo is a 100 baht replacement, not a warranty claim.

2.1.2 Weaknesses

- **Single point of failure.** If the ESP8266 reboots mid-cycle the meal is lost. There is no watchdog external to the board.
- **History lives in RAM.** A power cut erases the daily count. This is a known limitation and is listed in the requirements.
- **The enclosure is cardboard.** It is adequate for a demonstration and honest about being a prototype, but it is not cat-proof over months.
- **One cat, one hopper.** There is no identity and no separation if a second cat uses the device.

2.1.3 Opportunities

- **Presence-aware feeding.** The same ultrasonic sensor that measures the hopper level can detect an approaching cat, so one hardware decision serves two requirements.
- **Local-first pattern.** The firmware is a small, readable example of a device that serves its own UI, which is useful for other course projects and for the owner's own repairs.
- **Data without a database.** A short ring buffer in flash is enough to answer the owner's only question, which is whether the cat was fed today.

2.1.4 Threats

- **Servo current draw.** A stalled servo can pull enough current to brown out the board. This was observed during bring-up and is the main electrical risk.
- **Wi-Fi band mismatch.** The ESP8266 is 2.4 GHz only. A 5 GHz-only phone hotspot cannot be used.
- **Sensor false positives.** A reflective hopper wall can fool an infrared proximity sensor into reporting food that is not there. This is why ultrasonic ranging is chosen for the level measurement.
- **Scope pressure.** A five-day window invites feature creep. The scope rule in 1.4 exists for this reason.

2.2 Tool and Technology Review

Each decision below was made against at least one alternative. The rejected option is recorded so the reader can see the trade that was accepted.

Table 4. Technology decisions

Decision	Chosen	Rejected	Rationale
Microcontroller	NodeMCU v3, ESP8266	Arduino Uno with ESP-01, ESP32	Already owned. Has Wi-Fi and a 3.3 V regulator on board. The ESP32 is better but adds cost and no requirement needs it.
Actuator	Servo SG90	Stepper motor, solenoid	The servo holds an angle without power cycling and needs one PWM pin. A solenoid slams and needs a driver transistor.
Time source	SNTP over Wi-Fi	DS3231 real-time clock	No extra part, no battery, and the time is correct after every boot. An RTC drifts and needs a battery holder.
Transport	HTTP on the device	MQTT broker, Blynk	HTTP needs no broker and no account. Blynk hides the implementation behind a vendor widget, which removes the work the assessment is looking for.
Dashboard	Embedded HTML and JavaScript in PROGMEM	SPIFFS files, external CDN	One flash image, no filesystem, no internet dependency, and the page cannot lose its assets.
Presence sensing	HC-SR04 ultrasonic	FC-51 infrared proximity	Ultrasonic reports distance, which supports both presence and level. The FC-51 is binary and is fooled by a cardboard hopper wall.
Portion control	HX711 with 1 kg load cell	Open-loop flap timing	Timing assumes a constant flow rate. A load cell measures the actual mass, which allows the controller to correct.

2.3 Risk Analysis

Table 5. Risk register

ID	Risk	Likelihood	Impact	Mitigation
R-01	Servo stall browns out the board and reboots it mid-meal	High	High	Power the servo from the powerbank rail, not from 3V3. Add a 470 uF capacitor across VIN and GND. Test the worst case with servo_test.ino before integrating.
R-02	Flap stays open and the hopper empties	Medium	High	The close is driven from millis() in the main loop, not from a network callback, so a dropped client cannot strand the flap.
R-03	Clock is wrong or jumps backward after a resync	Medium	Medium	Ignore the schedule until the clock passes a fixed epoch. A duplicate meal inside the same minute is prevented by a minute key.
R-04	Ultrasonic readings oscillate on a kibble surface	High	Medium	Sample ten times per measurement and use the median. Reject echoes outside the hopper depth.
R-05	Load cell cannot be mounted rigidly on cardboard	Medium	Medium	Decide by 2 November. If the platform flexes, drop the feature and record it as Future Work.
R-06	Wi-Fi in the examination room blocks the device	Low	High	The dashboard is reachable in the local network, so the demonstration does not depend on internet access. A phone hotspot is the fallback.
R-07	Parts do not arrive before 30 October	Medium	High	The firmware already meets the schedule and dashboard requirements without any sensor. The sensors are additive, not load bearing.

Chapter 3 Product Perspective

3.1 Product Perspective

The device is a standalone appliance with a browser interface. It is not a component of a larger platform, and it does not report to a server. This choice is deliberate. A self-hosted device has a smaller attack surface, a smaller failure surface, and a demonstration that survives a broken internet connection.

Inside the appliance there are four responsibilities. The web layer turns HTTP requests into commands and status. The scheduler compares local time against a fixed table of meals. The actuation layer drives the servo. The state layer records what happened. The important design rule is that every feeding path, whether from the browser, the clock, or a sensor, passes through one function so that counting and safety rules cannot diverge.

3.2 Product Features

Table 6. Feature summary

ID	Feature	Actor	Status
F1	Scheduled feeding at 08:00, 13:00, and 18:00 using NTP time	System clock	Delivered
F2	On-demand feeding from the web dashboard	Owner	Delivered
F3	Flap status, daily count, last meal, and connection quality	Owner	Delivered
F4	Cat presence detection and presence-triggered meal	Cat	Parts received
F5	Portion control by weight, and hopper level	System	Parts received

3.3 System Architecture

The context diagram below shows every entity that the device exchanges energy or data with. Dashed elements are planned and are marked as such so the diagram does not claim hardware that is not on the bench.

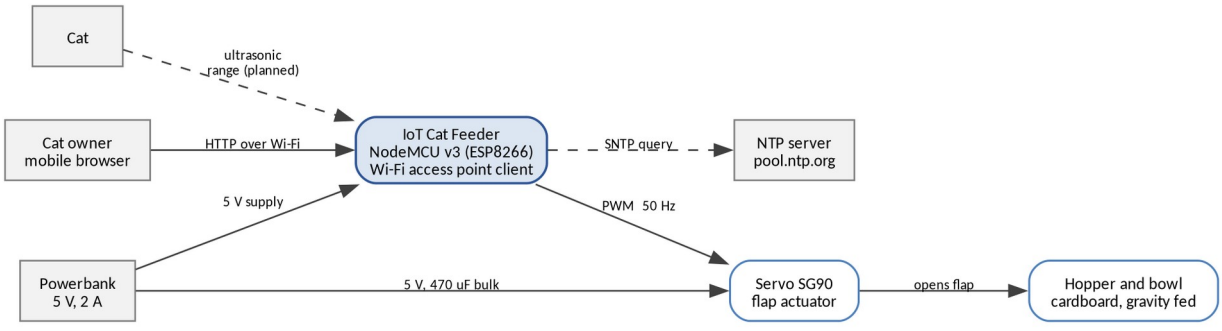


Figure 1. Context diagram of the proposed system.

3.4 Use Case Overview

Five use cases are identified. UC-01 to UC-03 are delivered. UC-04 and UC-05 are the presence and portion targets.

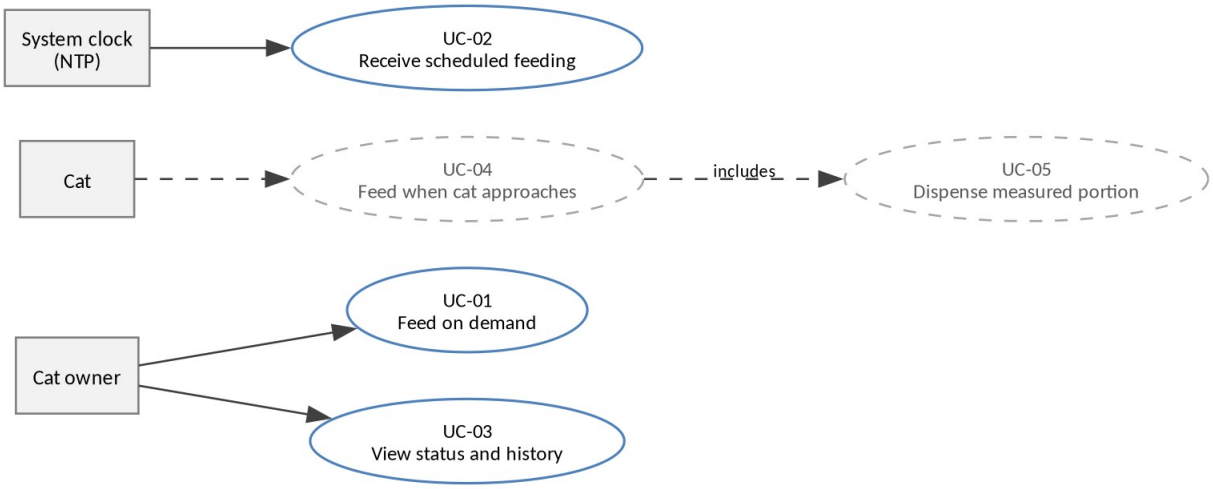


Figure 2. Use case diagram. Dashed use cases are planned.

Chapter 4 Quality Planning

Quality requirements are grouped by category and given identifiers so the design and the test plan can refer to them. The full text of each requirement is in the SRS.

Table 7. Quality requirement summary

Category	ID	Requirement in one line
Performance	NFR-PM-01	Dashboard responds within 2 seconds on the same network.
Performance	NFR-PM-02	Presence detection dispenses within 2 seconds of the cat entering range.
Reliability	NFR-RL-01	Flap never remains open longer than 3 seconds.
Reliability	NFR-RL-02	Device reconnects to Wi-Fi without a reboot.
Reliability	NFR-RL-03	Schedule works for 24 hours without drift beyond 1 minute.
Usability	NFR-UH-01	Dashboard is readable on a 375 px wide screen without zoom.
Usability	NFR-UH-02	A first-time user can feed the cat without instructions.
Security	NFR-SEC-01	Wi-Fi credentials are never committed to the repository.
Security	NFR-SEC-02	Only the documented HTTP methods are accepted.
Maintainability	NFR-OP-01	The firmware builds with the Arduino ESP8266 core and documented libraries.
Constraint	NFR-CP-01	Runs from a 5 V, 2 A powerbank with no laptop attached.
Constraint	NFR-CP-02	Operates at 2.4 GHz Wi-Fi only.

Chapter 5 Limits, Milestones, and Deliverables

5.1 Limits

- The submission deadline is 4 November 2026. Any feature not tested by 2 November is descoped.
- The build is limited to parts already owned plus a sensor budget of about 500 baht.
- The enclosure is built from cardboard and a food-safe bowl. Machining and 3D printing are not available in the time window.
- The device runs on a single board with no server, no database, and no cloud account.

5.2 Milestones

Table 8. Milestones

Date	Milestone	Exit criterion
30 Oct 2026	Documents and parts order	Proposal, SRS, and SDD drafted. Sensors ordered.
31 Oct 2026	Requirements frozen	Requirement set baselined in version control.
1 Nov 2026	Presence detection integrated	Cat detected in 16 of 20 trials.
2 Nov 2026	Portion control integrated or descoped	Weight within 15 percent, or feature removed.
3 Nov 2026	System test and evidence	All test cases executed. Photos and video captured.
4 Nov 2026	Submission	Repository tagged, report submitted.

5.3 Deliverable Items

1. The assembled feeder, working from a powerbank.
2. Firmware source in firmware/feeder/feeder.ino and the servo bring-up sketch in firmware/servo_test/servo_test.ino.
3. Wiring diagram in docs/wiring_nodemcu_servo.html.
4. This proposal, the SRS, and the SDD.
5. Test records with photographs and a demonstration video.
6. The output and discussion chapter with measured results.
7. The conclusion and future work chapter.

5.4 Budget

Prices are estimates from general retail listings and are not yet confirmed with the supplier.

Table 9. Estimated bill of materials

Item	Quantity	Unit (baht)	Total (baht)
NodeMCU v3 (already owned)	1	0	0
Servo SG90 (already owned)	1	0	0
Capacitor 470 uF 16 V	1	10	10
HC-SR04P ultrasonic sensor	2	80	160
HX711 amplifier with 1 kg load cell	1	160	160
Breadboard, jumpers, resistor (already owned)	1 set	0	0
Cardboard, tape, food-safe bowl	1 set	50	50

Item	Quantity	Unit (baht)	Total (baht)
Powerbank 5 V 2 A (already owned)	1	0	0
Total			**380**

5.5 Future Work

- Move the daily history into flash so it survives a power cut.
- Add an ESP32-CAM module for a still image per meal, so the owner can confirm the cat ate rather than the bowl.
- Identify the cat with an RFID collar tag so a second cat cannot take the portion.
- Add a hopper level alarm that publishes to the dashboard instead of only showing a value.
- Replace the cardboard enclosure with a printed or laser-cut body.